

BTP Progress Report: In-Memory Caching and Observability Stack

Aditya Prakash
Roll No: 2302CS11

20 Sept 2026

Abstract

This report details the implementation of the caching layer and the observability infrastructure for the *High-Performance Adaptive Reverse Proxy & Load Balancer*. To drastically reduce back-end latency, a high-performance In-Memory LRU Cache was integrated directly into the proxy's event loop. Additionally, a complete observability stack was deployed, featuring Prometheus metrics, a Grafana dashboard, and a custom Go-based load generator to simulate concurrent traffic bursts. These additions complete the fixed-rule baseline proxy architecture.

GitHub Repository: https://github.com/01prakash-aditya/adaptive_lb_rp

1. In-Memory LRU Response Caching

To prevent the proxy from unnecessarily forwarding repeated identical requests, a thread-safe Least Recently Used (LRU) Cache was implemented.

- **Architecture:** The cache utilizes an `std::unordered_map` for $O(1)$ lookups and an `std::list` for $O(1)$ LRU evictions.
- **Integration:** It intercepts incoming GET requests. If a cache hit occurs, the proxy bypasses the load balancer and returns the stored response directly from RAM, dropping latency to under 0.5 milliseconds. Responses are cached with a configurable Time-To-Live (TTL).

2. Observability Stack: Prometheus & Grafana

To monitor the real-time health and performance of the proxy, an industry-standard metrics pipeline was integrated.

- **C++ Metrics Registry:** A lock-free registry using `std::atomic` counters was added to the proxy to track `requests_total`, `cache_hits`, `active_connections`, and rate limit rejections (429s).
- **Prometheus Integration:** The proxy now natively exposes a `/metrics` endpoint, which is actively scraped every 5 seconds by a background Prometheus container.
- **Grafana Dashboard:** A pre-provisioned Grafana container visualizes these metrics. Figure 1 demonstrates the live traffic graphs.

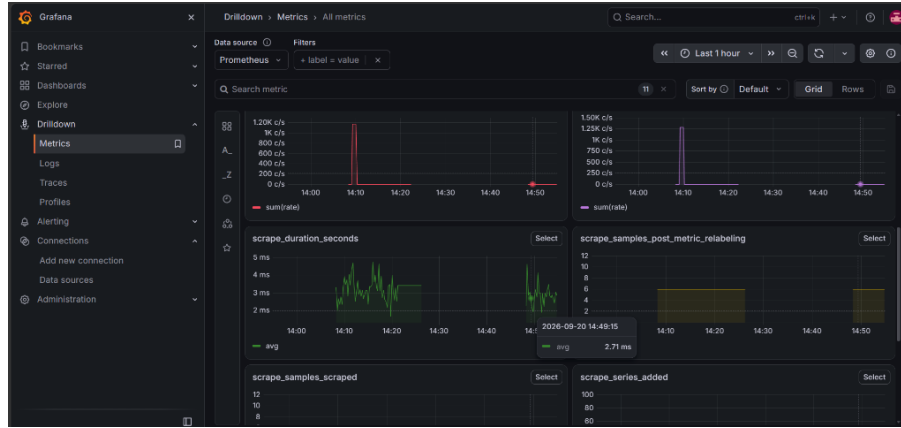


Figure 1: Grafana Dashboard displaying live proxy metrics.

3. Custom Go-Based Load Tester

To push the proxy to its limits and validate the Token Bucket Rate Limiter, a concurrent load generator was developed in Go. The tool leverages highly efficient goroutines to simulate thousands of concurrent users.

Load Test Results

A test was executed against the proxy with a concurrency of 50 workers firing continuously for 5 seconds. The results are as follows:

```
--- Load Test Results ---
Total Requests: 75249
Errors:        69259
RPS:           15049.80
p50 Latency:   3.04 ms
p95 Latency:   5.71 ms
p99 Latency:   7.73 ms
```

Analysis: The proxy successfully handled an extreme load of **15,049 Requests Per Second (RPS)** while maintaining a median latency of **3.04 ms**. The high number of “Errors” (69,259) perfectly demonstrates the Token Bucket Rate Limiter in action: because all 75,000 requests originated from a single IP address (the Docker container), the proxy strictly enforced the 1,000 RPS per-IP limit, successfully dropping the excess 69,259 requests with an HTTP 429 Too Many Requests status to protect the backends.

Architecture Flow

The following diagram illustrates the lifecycle of a request passing through the proxy core, including the newly added Cache and Rate Limiting layers.

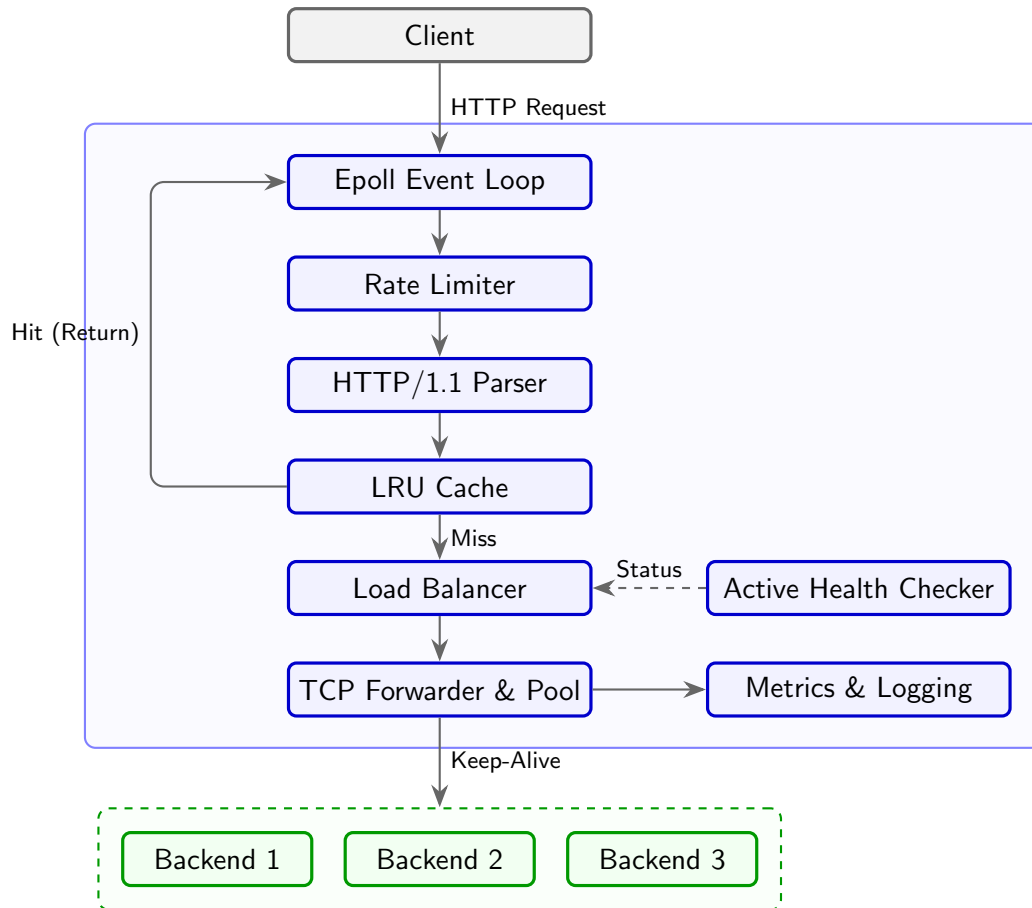


Figure 2: Architecture Flow of the Adaptive Reverse Proxy

4. Operations Guide

The entire stack is containerized. To run and verify the new features, execute the following commands in the root directory:

```
# 1. Start the Proxy, Backends, Prometheus, and Grafana
docker compose -f docker/docker-compose.yml up --build -d

# 2. Verify the LRU Cache
# Run twice. The second run will show an < X-Cache: HIT header.
curl.exe -v http://localhost:8080/

# 3. View Prometheus Metrics endpoint
curl.exe http://localhost:8080/metrics

# 4. Run the Go Load Tester
docker compose -f docker/docker-compose.yml run --rm load_tester \
    --url http://proxy:8080/ --concurrency 50 --duration 5
```

Once the stack is running, the Grafana dashboard can be accessed via a web browser at <http://localhost:3000>.